

```
In [1]: a = 5  
a
```

```
Out[1]: 5
```

```
In [2]: a = 5.5  
type(a)
```

```
Out[2]: float
```

```
In [3]: a = 5.5  
print(type(a))
```

```
<class 'float'>
```

```
In [4]: b = [2,3]  
print(type(b))
```

```
<class 'list'>
```

```
In [5]: class laptop(): # class name  
        def config(): #method name  
            print('i5', 16) # print statment  
            print('i9', 32)
```

```
lap = laptop() #lap is object
```

```
In [6]: class laptop(): # class name  
        def config(): #method name  
            print('i5', 16) # print statment  
            print('i9', 32)
```

```
lap = laptop() #lap is object
```

```
laptop.config()
```

```
i5 16
```

```
i9 32
```

```
In [7]: class laptop(): # class name  
        def config(): #method name  
            print('i5', 16) # print statment  
            print('i9', 32)
```

```
#lap = laptop() #lap is object
```

```
laptop.config()
```

```
i5 16
```

```
i9 32
```

```
In [8]: class laptop(): # class name  
        def config(): #method name  
            print('i5', 16) # print statment  
            print('i9', 32)
```

```
lap = laptop() #lap is object

lap.config()
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[8], line 8
      4         print('i9', 32)
      6 lap = laptop() #lap is object
----> 8 lap.config()

TypeError: laptop.config() takes 0 positional arguments but 1 was given
```

```
In [9]: class laptop(): # class name
        def config(self): #method name
            print('i5', 16) # print statment
            print('i9', 32)

lap = laptop() #lap is object

lap.config()
```

```
i5 16
i9 32
```

```
In [10]: class laptop(): # class name
        def config(self): #method name
            print('i5', 16) # print statment
            print('i9', 32)

lap1 = laptop() #lap is object
lap2 = laptop()
lap3 = laptop()
lap4 = laptop()

lap1.config()
lap2.config()
lap3.config()
```

```
i5 16
i9 32
i5 16
i9 32
i5 16
i9 32
```

```
In [11]: class computer():          # computer is class (user create the class hear(user defin
        def config():              # confit is method (define some method hear so when we talk
            print('i5', '8gb') # i want to print configuration of the machine which i5

a = 8
print(type(a))

com1 = computer() #com1 is the user define object of the computer class but if you
print(type(com1))
```

```
<class 'int'>
<class '__main__.computer'>
```

In [12]: `__name__`

Out[12]: `'__main__'`

```
In [13]: class computer:
          def config():      # self is the object which user passing
              print('i5', '8gb')

          com1 = computer()

          computer.config() #hear com1 act as parameter going to the self
```

i5 8gb

```
In [14]: class computer():
          def config():      # self is the object which user passing
              print('i5', '8gb')

          com1 = computer()

          computer.config(com1) #hear com1 act as parameter going to the self
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[14], line 7
      3     print('i5', '8gb')
      5 com1 = computer()
----> 7 computer.config(com1)

TypeError: computer.config() takes 0 positional arguments but 1 was given
```

```
In [15]: class computer:
          def config(self):    # self is the object which user passing
              print('i5', '8gb')

          com1 = computer()

          computer.config(com1) #hear com1 act as parameter going to the self
```

i5 8gb

```
In [16]: class computer:
          def config(self):    # self is the object which user passing & it is the keyw
              print('i5', '8gb')

          com1 = computer()

          com1.config()
```

i5 8gb

Special Method

```
In [17]: class computer():
          def __init__(self):
              print('i7', '16gb')

          c1 = computer()

          #c1.__init__()
```

i7 16gb

```
In [18]: class computer():
          def __init__(self):
              print('i7', '16gb')

          c1 = computer()
          c3 = computer()
          c3 = computer()

          #c1.__init__()
```

i7 16gb

i7 16gb

i7 16gb

```
In [19]: class computer():
          def config(self):
              print('i7', '16gb')

          c1 = computer()
          c2 = computer()
          c3 = computer()

          c1.config()
          c2.config()
```

i7 16gb

i7 16gb

```
In [20]: class computer():

          def __init__(self):
              print('in init')

          # def config(self):
          #     print('i5', '8gb')

          com1 = computer()
          #com2 = computer()

          #com1.config()
          #com2.config()
```

in init

```
In [21]: class computer():

          def __init__(self):
```

```

        print('in init')

    def config(self):
        print('i5', '8gb')

com1 = computer()
#com2 = computer()

com1.config()
#com2.config()

```

```

in init
i5 8gb

```

```

In [22]: class computer():

    def __init__(self):
        print('in init')

    def config(self):
        print('i5', '8gb')

com1 = computer()
#com2 = computer()

com1.__init__()
com1.config()

#com2.config()

```

```

in init
in init
i5 8gb

```

```

In [23]: class computer:

    def config(self):
        print('i5', '8gb')

    def config2(self):
        print('i7', '32gb')

    def __init__(self):
        print('in init')

com1 = computer()
com2 = computer()
com3 = computer()

com1.config() # we didnt pass any argument but system bydefault consider self as an
com1.config2()
#com2.config1()

```

```

in init
in init
in init
i5 8gb
i7 32gb

```

```

In [24]: class computer():

    def __init__(self):
        print('in init')

    def config(self):
        print('i5', '8gb')

com1 = computer('i5', 8) # com1 has cpu i5 & ram is 8
com2 = computer('i7', 16) # com2 has cpu intel 5 & ram is 8 as we have 2 computer w

com1.config()
com2.config()

```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[24], line 9
      6     def config(self):
      7         print('i5', '8gb')
----> 9 com1 = computer('i5', 8) # com1 has cpu i5 & ram is 8
     10 com2 = computer('i7', 16) # com2 has cpu intel 5 & ram is 8 as we have 2 com
puter we have different ram & cpt
     12 com1.config()

TypeError: computer.__init__() takes 1 positional argument but 3 were given

```

```

In [25]: class computer:

    def __init__(self,cpu,ram): # cpu & ram as argument
        print('in init')

    def config(self):
        print('i5', '8gb')
        print('i7', 16)

com1 = computer('i5', 8, 10000)
com2 = computer('i7', 16, 20000)

com1.config()
com2.config()

```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[25], line 10
      7     print('i5', '8gb')
      8     print('i7', 16)
----> 10 com1 = computer('i5', 8, 10000)
      11 com2 = computer('i7', 16, 20000)
      13 com1.config()

TypeError: computer.__init__() takes 3 positional arguments but 4 were given
```

```
In [26]: class computer:

    def __init__(self,cpu,ram, price): # cpu & ram as argument
        print('in init')

    def config(self):
        print('i5', '8gb', 10000)
        print('i7', 16, 2000)

com1 = computer('i5', 8, 10000)
com2 = computer('i7', 16, 20000)

com1.config()
com2.config()
```

```
in init
in init
i5 8gb 10000
i7 16 2000
i5 8gb 10000
i7 16 2000
```

```
In [27]: class computer():

    def __init__(self,cpu,ram): # cpu & ram as argument
        self.cpu = cpu #hear self is the object & we need to refer to the object
        self.ram = ram

    def config(self):
        print('config is', self.cpu, self.ram)

com1 = computer('i5', 8)
com2 = computer('i7', 16)

com1.config()
com2.config()

#print(id(com1))
#print(id(com2))
```

```
config is i5 8
config is i7 16
```

```
In [28]: class computer():

    def __init__(self,cpu,ram, price, company): # cpu & ram as argument
```

```

        self.cpu = cpu #hear self is the object & we need to refer to the object
        self.ram = ram
        self.price = price
        self.company = company

    def config(self):
        print('config is', self.cpu, self.ram, self.price, self.company)

com1 = computer('i5', 8, 50000, 'hp')
com2 = computer('i7', 16, 80000, 'lenovo')

com1.config()
com2.config()

#print(id(com1))
#print(id(com2))

```

```

config is i5 8 50000 hp
config is i7 16 80000 lenovo

```

In [29]: `class computer():` *#computer class & every concept has a object or*

```

c1 = computer() #c1 is object which referring to clas
print(id(c1))

```

Cell In[29], line 3

```

c1 = computer() #c1 is object which referring to clas
^

```

IndentationError: expected an indented block after class definition on line 1

In [30]: `class computer:` *#computer class & every concept has a object or*
`pass`

```

c1 = computer() #c1 is object which referring to clas
c2 = computer()

print(id(c1))
print(id(c2))

```

```

2645479410096

```

```

2645479407104

```

constructor & self

In [31]: `class computer():`
 `def __init__(self):`
 `self.name = 'akash'`
 `self.age = 37`

```

c1 = computer()
c2 = computer()

print(c1.name, c1.age)
print(c2.name)

```


akash 37
akash

```
In [32]: class computer():
          def __init__(self):
              self.name = 'akash'
              self.age = 37

          c1 = computer()
          c2 = computer()

          c2.name = 'mit'
          c1.age = 38
          c1.name = 'rakesh'
          c2.age = 40

          print(c1.name, c1.age)
          print(c2.name, c2.age)
```

rakesh 38
mit 40

```
In [33]: class computer():
          def __init__(self):
              self.name = 'akash'
              self.age = 37

          c1 = computer()
          c2 = computer()

          c2.name = 'mit'
          c1.age = 38
          c1.name = 'rakesh'
          c2.age = 40

          print(c1.name, c1.age)
          print(c2.name, c2.age)

          print(id(c1))
          print(id(c2))
```

rakesh 38
mit 40
2645479435984
2645479439008

encapsulation

- variables

```
In [34]: class car:
          def __init__(self):
              self.mil = 10
              self.com = 'bmw'
```

```
c1 = car()
c2 = car()

print(c1.com, c1.mil)
print(c2.com, c2.mil)

print(id(c1))
print(id(c2))
```

```
bmw 10
bmw 10
2645479440256
2645476812432
```

```
In [35]: class car():
        def __init__(self):
            self.mil = 10
            self.com = 'bmw'

        c1 = car()
        c2 = car()
        c3 = car()

        c2.mil = 8

        c1.mil = 20
        c1.com = 'Maruti'

        print(c1.com, c1.mil)
        print(c2.com, c2.mil)

        print(c3.com)
```

```
Maruti 20
bmw 8
bmw
```

```
In [36]: class car:
        def __init__(self):
            self.mil = 10
            self.com = 'bmw'

        c1 = car()
        c2 = car()

        c2.mil = 12

        print(c1.com, c1.mil)
        print(c2.com, c2.mil)
```

```
bmw 10
bmw 12
```

```
In [37]: class car():

        wheels = 5
```

```

def __init__(self):
    self.mil = 10
    self.com = 'bmw'

c1 = car()
c2 = car()

c2.mil = 8

print(c1.com, c1.mil, c1.wheels)
print(c2.com, c2.mil, c2.wheels)

```

bmw 10 5

bmw 8 5

In [38]:

```

class car(): # car is class

    wheels = 5    # class variable

    def __init__(self): # special method
        self.mil = 10 # instance variance (mil)
        self.com = 'bmw' # instance variable (com)

c1 = car() #c1 is object referece to class
c2 = car() # c2 is obejct refere to car

c1.wheels= 6    #object.class varieble
c2.wheels = 3 #object.class variabele

c2.mil = 8 # obj with instance variable

print(c1.com, c1.mil, c1.wheels)
print(c2.com, c2.mil, c2.wheels)

```

bmw 10 6

bmw 8 3

In []:

In []: